

Anexo — Videos de apoyo

Manual del curso JavaScript Intermedio · REDEC-UNAM / FESC · Educación Continua

Propósito

Este anexo reúne una selección de videos en YouTube que complementan, con lenguaje directo y sin exceso de tecnicismos, cada tema del manual. Cada ficha relaciona el tema con un **caso de uso real** y una **pregunta para pensar**, con el fin de favorecer el aprendizaje significativo: entender para qué sirve cada concepto antes de memorizar su sintaxis.

Lista de reproducción del curso

Todos los videos están organizados en una lista de reproducción:

<https://youtube.com/playlist?list=PLBN8bJQ3f4w4&si;=EaNHFKV2oJEhtuP1>

Cómo usar las fichas

1. Antes de cada video, intenta predecir qué va a pasar.
2. Mira el video **Principal**; los de **Refuerzo** sirven para repasar o como tarea previa.
3. Relaciona el tema con el caso de uso y responde la pregunta.
4. Los videos marcados como *opcional (inglés)* se pueden ver con subtítulos.

Contenido

Sesión	Tema	Fecha	Pág.
1	Fundamentos avanzados: Event Loop, scope, closures y this	28 sep 2026	2
2	POO y ES6+: prototipos, clases, desestructuración y módulos	29 sep 2026	4
3	Programación asíncrona: callbacks, promesas, async/await y Fetch	30 sep 2026	6
4	DOM, eventos, Web Storage y formularios	1 oct 2026	8
5	Testing, depuración y arquitectura profesional	2 oct 2026	10

Nota: los enlaces pueden cambiar o dejar de estar disponibles con el tiempo; se recomienda verificarlos antes de cada edición del curso.

Nota de divulgación

Este material fue elaborado con asistencia de Claude (Anthropic) y revisado por Jesús Armando Tapia Gallegos.

Videos de apoyo — JavaScript Intermedio

Sesión 1: Fundamentos Avanzados · REDEC-UNAM / FESC · 28 de septiembre de 2026

Cómo usar esta ficha: antes de ver cada video, intenta predecir qué va a pasar. Después, piensa en el caso de uso y responde la pregunta. Recuerda: *no buscamos sólo código que funcione, sino código que podamos explicar.*

1.1 · Call Stack y Event Loop

Idea clave: JavaScript hace una cosa a la vez; el navegador espera por él.

Tipo	Video (clic para abrir)
Principal	La pila de ejecución (Call Stack) – JS en Español
Principal	Entiende el Event Loop de JavaScript en 10 minutos
Refuerzo	Qué es el Event Loop en JavaScript – paso a paso
Opcional (inglés)	What the heck is the event loop anyway? – Philip Roberts

Caso de uso real: Un botón de «Descargar reporte» sigue respondiendo mientras llegan los datos del servidor: la página no se congela.

Pregunta para pensar: ¿Por qué `setTimeout(fn, 0)` no se ejecuta «inmediatamente»?

1.2 · Hoisting y Scope: var, let, const

Idea clave: ¿Dónde «vive» cada variable y desde cuándo existe?

Tipo	Video (clic para abrir)
Principal	VAR, LET o CONST: ¿cuál debería usar? – JS en Español
Refuerzo	Diferencia entre var, let y const – Curso JS desde cero #4
Opcional (inglés)	Scope vs Closure vs Hoisting

Caso de uso real: Una lista de productos con botón «Agregar»: con `var` todos los botones agregan el mismo producto.

Pregunta para pensar: ¿Por qué cambiar `var` por `let` en el ciclo corrige el error?

1.3 · Closures

Idea clave: Una función puede «recordar» el lugar donde nació.

Tipo	Video (clic para abrir)
Principal	Closures en JavaScript: qué son y cómo funcionan
Casos reales	3 ejemplos REALES de Closures en JavaScript
Refuerzo	¿Qué son los Closures y por qué dan tanto miedo?

Caso de uso real: El saldo de una cuenta bancaria que sólo cambia con `depositar()` y `retirar()`, nunca «desde afuera».

Pregunta para pensar: ¿Por qué la cuenta de Ana y la de Luis no comparten saldo?

1.4 · this, call, apply y bind

Idea clave: this depende de *quién llama* a la función, no de dónde se escribió.

Tipo	Video (clic para abrir)
Principal	This en JavaScript (bind, call, apply y más)
Práctico	¿Para qué sirve call(), bind() y apply()?
Refuerzo	¿Cómo usar call, apply y bind? This a profundidad

Caso de uso real: Un temporizador que «olvida» a qué objeto pertenece cuando se pasa como callback a setInterval.

Pregunta para pensar: ¿Cuándo usarías bind y cuándo una arrow function para conservar el contexto?

Cierre: ¿cómo se conecta todo?

Tiempo (Event Loop) · **Espacio** (Scope) · **Contexto** (this) → los **Closures** son la memoria que une estos tres.

Actividad 1: entrega un archivo .js con los ejercicios 1.1 a 1.4. Cada bloque debe incluir comentarios que expliquen qué hace, por qué funciona y qué concepto demuestra. Entrégalo antes de la Sesión 2.

Videos de apoyo — JavaScript Intermedio

Sesión 2: POO y ES6+ · REDEC-UNAM / FESC · 29 de septiembre de 2026

Cómo usar esta ficha: antes de ver cada video, intenta predecir qué va a pasar. Después, piensa en el caso de uso y responde la pregunta. Recuerda: *código moderno no significa más sintaxis, sino una intención más clara*.

2.1 · Estructura moderna: template literals, arrow functions y parámetros por defecto

Idea clave: ES6+ no es un JavaScript nuevo: es el mismo lenguaje, más expresivo.

Tipo	Video (clic para abrir)
Principal	Curso de ES6 04 – Template literals o literales de texto
Refuerzo	Template strings en JavaScript (plantillas literales)
Opcional (inglés)	ES6+: Arrow Functions, Template Literals & Destructuring

Caso de uso real: Un mensaje de ticket: «Producto: Laptop · IVA: \$2,960» en una sola línea legible, y un usuario nuevo que recibe el rol «invitado» si no se indica otro.

Pregunta para pensar: ¿Por qué una arrow function no es sólo «una función más corta»? (Pista: this)

2.2 · Prototipos: el motor oculto

Idea clave: Muchos objetos comparten comportamiento sin copiarlo: lo buscan en su prototipo.

Tipo	Video (clic para abrir)
Principal	Herencia prototípica en JavaScript (POO)
Refuerzo	Curso JavaScript: 23. Herencia prototípica – jonmircha
Profundidad	Prototypes a profundidad – herencia prototípica

Caso de uso real: 1,000 empleados comparten los métodos calcularBono() y mostrarPerfil() en un «catálogo común», en lugar de tener 1,000 copias.

Pregunta para pensar: ¿Dónde está definido hacerSonido: dentro de perro o en Animal.prototype?

2.3 · Clases ES6: constructor, getters/setters, privados, extends y super

Idea clave: class es una fachada legible sobre los prototipos, no un modelo distinto.

Tipo	Video (clic para abrir)
Principal	Las clases y sus métodos: constructor, getters y setters – JS desde cero #12
Herencia	Herencia de clases: extends y super – JS desde cero #13
Refuerzo	Curso JavaScript: 25. Métodos estáticos, getters y setters – jonmircha

Caso de uso real: Un sistema de nómina donde el #salario es privado y nadie puede asignarle un valor negativo; Gerente y Desarrollador heredan de Empleado.

Pregunta para pensar: ¿Por qué calcularNomina() tiene sentido como método estático?

2.4 · Desestructuración, Spread y Rest

Idea clave: Describe qué datos quieres, no cómo sacarlos paso a paso. Los tres puntos (...) expanden o agrupan según dónde estén.

Tipo	Video (clic para abrir)
Principal	JavaScript moderno: Desestructuración y Operador Spread
Refuerzo	Curso JavaScript Moderno (ES6) #16 – Destructuring
Rest	Desestructuración avanzada en JavaScript: parámetros Rest

Caso de uso real: Limpiar la respuesta de una API: sacar nombre, ciudad y teléfono, y crear copias con activo: true sin alterar los datos originales.

Pregunta para pensar: Si copio un objeto con spread y cambio su dirección anidada, ¿se modifica también el original? ¿Por qué?

2.5 · Módulos ESM: import / export

Idea clave: Cada archivo tiene una responsabilidad y sólo expone lo que otros necesitan.

Tipo	Video (clic para abrir)
Principal	Import y Export en JavaScript (ES Modules) – Explicación
Refuerzo	Javascript #14: Módulos (export, export default, import)
Refuerzo	Curso JavaScript: 33. Módulos (import / export) – jonmircha

Caso de uso real: Dividir una tienda en línea en productos.js, carrito.js y app.js: si falla el cálculo del total, sabes exactamente dónde buscar.

Pregunta para pensar: ¿Cuándo importas con llaves { } y cuándo sin ellas?

Cierre: ¿cómo se conecta todo?

Prototipos (el motor) → **Clases** (el modelo) → **Desestructuración / Spread / Rest** (los datos) → **Módulos** (la arquitectura). La Sesión 2 no reemplaza la Sesión 1: usa this, scope y closures.

Actividad 2: en un mismo repositorio, entrega la jerarquía Empleado / Gerente / Desarrollador (ejercicio 2.2) y el carrito dividido en módulos (ejercicio 2.4), con una estructura de carpetas clara. Se evalúa si cada archivo deja ver qué responsabilidad tiene, no sólo si el código corre.

Videos de apoyo — JavaScript Intermedio

Sesión 3: Programación Asíncrona · REDEC-UNAM / FESC · 30 de septiembre de 2026

Cómo usar esta ficha: antes de ver cada video, intenta predecir qué va a pasar. Después, piensa en el caso de uso y responde la pregunta. Hoy la pregunta clave es: *¿cuándo empieza esta operación, cuándo termina y qué pasa si falla?*

3.1 · Operaciones no bloqueantes

Idea clave: JavaScript puede iniciar algo que termina después y seguir trabajando mientras tanto.

Tipo	Video (clic para abrir)
Principal	¿Qué es código asíncrono y síncrono en JavaScript? (con ejemplos simples)
Refuerzo	¿Qué es la programación asíncrona en JavaScript? Explicación fácil
Visual	JavaScript asíncrono: síncrono vs. asíncrono – descripción gráfica

Caso de uso real: El mesero de un restaurante no se queda parado frente a la cocina: toma otras órdenes y regresa cuando el platillo está listo.

Pregunta para pensar: ¿Por qué «La aplicación continúa» aparece antes que «Operación terminada»?

3.2 · Callbacks y Callback Hell

Idea clave: Un callback es «llámame cuando termines». Útil, pero anidado se vuelve una pirámide difícil de mantener.

Tipo	Video (clic para abrir)
Principal	Asincronismo en JS: ¿qué es el Callback Hell? – JS desde cero #15
Refuerzo	Callbacks: ¿qué son y cómo utilizarlos? – Evitar callback hell
Refuerzo	Qué es el antipatrón Callback Hell en JavaScript

Caso de uso real: Procesar fotos: leer archivo → medir → redimensionar → guardar. Cada paso anidado dentro del anterior, con el manejo de errores repetido en cada nivel.

Pregunta para pensar: ¿Qué tan fácil sería agregar un quinto paso a esa pirámide?

3.3 · Promesas: then, catch, finally, Promise.all y race

Idea clave: Una Promise es un «comprobante»: la gestión está pendiente y terminará una sola vez, con éxito o con error.

Tipo	Video (clic para abrir)
Principal	Promesas: new Promise, resolve, reject, then, catch
Refuerzo	Cómo usar promesas en JavaScript – JS en Español
Paralelo	Promesas: métodos Promise.all y race en JavaScript

Caso de uso real: Un panel que carga usuario, productos y categorías a la vez con Promise.all: tarda lo que la más lenta, no la suma de las tres.

Pregunta para pensar: ¿Cuándo NO conviene usar Promise.all? (Pista: dependencias)

3.4 · Async / Await y try / catch

Idea clave: Se lee de arriba hacia abajo, pero sigue siendo asíncrono: await pausa sólo esa función, no toda la página.

Tipo	Video (clic para abrir)
Principal	Promesas y async-await: then y catch – JS desde cero #16
Comparativo	Callback, Promesas, Async Await y Try Catch JavaScript
Refuerzo	JavaScript asíncrono con Async Await

Caso de uso real: Iniciar sesión: esperar la validación del usuario y después cargar su perfil, con un solo try/catch que avisa si algo falla.

Pregunta para pensar: ¿Qué cambió entre callback, Promise y async/await: la operación o la forma de escribir el flujo?

3.5 · Fetch API y manejo de errores

Idea clave: Que el servidor conteste no significa que todo salió bien: siempre revisa response.ok.

Tipo	Video (clic para abrir)
Principal	Cómo consumir una API REST con Fetch + Promises con gestión de errores
Errores	Curso de JavaScript: API fetch – manejo de errores
Refuerzo	Javascript Fetch API: qué es y cómo consumir un API

Caso de uso real: Una lista de usuarios que muestra «Cargando...», enseña los datos o un mensaje claro si hay 404 o no hay conexión, y siempre quita el indicador al final (finally).

Pregunta para pensar: Si el servidor responde 404, ¿fetch() entra al catch por sí solo? ¿Por qué?

Cierre: ¿cómo se conecta todo?

No bloquear → **Callbacks** (solución histórica) → **Promesas** (cadena plana, errores centralizados) → **Async/Await** (lectura secuencial) → **Fetch** (datos reales, validando HTTP). Conecta con la Sesión 1: microtarear antes que setTimeout.

Actividad 3: construye un cliente API en tres versiones comentadas: callbacks, Promises y async/await. Debe distinguir error de red y error HTTP, y explicar por qué la versión final es más legible y fácil de mantener.

Videos de apoyo — JavaScript Intermedio

Sesión 4: DOM, eventos, persistencia y formularios · REDEC-UNAM / FESC · 1 de octubre de 2026

Cómo usar esta ficha: antes de ver cada video, intenta predecir qué va a pasar. Después, piensa en el caso de uso y responde la pregunta. Idea del día: *actualiza sólo lo necesario, escucha desde el lugar adecuado, conserva el estado que corresponde y valida antes de enviar.*

4.1 · Interacción eficiente: DocumentFragment

Idea clave: Prepara todo en una «mesa de trabajo» y entrégalo al DOM de una sola vez.

Tipo	Video (clic para abrir)
Principal	DOM – Curso JavaScript Moderno #04: Fragment + createElement
Breve	DocumentFragment en JavaScript (short)
Opcional (inglés)	What is reflow & repaint in the browser?

Caso de uso real: Mostrar un catálogo de 500 productos traídos de una API: se arma la lista completa en memoria y se inserta en la página en un solo paso.

Pregunta para pensar: ¿Qué parte del proceso ocurre fuera de lo que el usuario ve?

4.2 · Propagación y delegación de eventos

Idea clave: Un solo listener en el contenedor atiende a todos los hijos, incluso a los que se agreguen después.

Tipo	Video (clic para abrir)
Principal	Delegación de eventos en JavaScript con ejemplo práctico
Bubbling	¿Cómo funciona el Event Bubbling? – JavaScript DOM
Refuerzo	¿Qué es y para qué sirve la delegación de eventos en JavaScript?

Caso de uso real: Una lista de tareas donde cada una tiene botones «Completar» y «Eliminar»: la lista escucha los clics y `closest('li')` identifica qué tarea se tocó.

Pregunta para pensar: Si agrego una tarea nueva después, ¿en qué momento se registró su listener?

4.3 · Web Storage: localStorage, sessionStorage y JSON

Idea clave: Una variable se pierde al recargar; Web Storage guarda texto, así que los objetos viajan con `JSON.stringify` y `JSON.parse`.

Tipo	Video (clic para abrir)
Principal	LocalStorage y SessionStorage ¿cómo funcionan? – Curso JavaScript #48
JSON	Guarda objetos y arrays en LocalStorage como un PRO (JSON explicado fácil)
Refuerzo	API localStorage – guardar y recuperar objetos con formato JSON

Caso de uso real: Tus tareas y el «modo oscuro» siguen ahí al presionar F5. Pero contraseñas o tokens nunca se guardan ahí: `localStorage` no es una caja fuerte.

Pregunta para pensar: ¿Qué pasa si guardas un arreglo en `localStorage` sin convertirlo a JSON?

4.4 · Formularios: preventDefault, FormData y validación

Idea clave: Validar no es impedir: es ayudar al usuario a producir datos que la aplicación pueda usar.

Tipo	Video (clic para abrir)
Principal	FormData en JavaScript: cómo, cuándo y por qué usarlo
preventDefault	preventDefault en JavaScript: evitando el comportamiento por defecto
Validación	Aprende a validar formularios con JavaScript y expresiones regulares

Caso de uso real: Un formulario de contacto que muestra todos los errores junto a cada campo (sin alert) y, si todo está bien, genera un objeto listo para enviar.

Pregunta para pensar: ¿Por qué conviene juntar todos los errores en un arreglo en lugar de detenerse en el primero?

Cierre: ¿cómo se conecta todo?

Datos → **DocumentFragment** (pintado óptimo) → **DOM** → **Delegación** (escucha inteligente) → **Estado** ↔ **Web Storage** (persistencia). Y por otro lado: **Formulario** → **FormData** → **Validación** → datos listos.

Actividad 4 (individual): lista de tareas con un solo listener delegado, persistencia en localStorage (guardar, cargar y limpiar) y un formulario validado con FormData, mostrando los errores en la interfaz (sin alert).

Videos de apoyo — JavaScript Intermedio

Sesión 5: Testing, Depuración y Arquitectura · REDEC-UNAM / FESC · 2 de octubre de 2026

Cómo usar esta ficha: antes de ver cada video, intenta predecir qué va a pasar. Después, piensa en el caso de uso y responde la pregunta. Idea del día: *el código funcional resuelve el problema de hoy; el código robusto está preparado para los cambios de mañana.*

5.1 · Testing: la pirámide de pruebas

Idea clave: Muchas pruebas pequeñas y rápidas en la base (unitarias), menos de integración y pocas de flujo completo (E2E).

Tipo	Video (clic para abrir)
Principal	Aprende en 3 minutos: pruebas unitarias y pruebas de integración
Refuerzo	Pruebas unitarias y de integración – Curso de Tester de Software
Opcional (inglés)	Testing Pyramid – Unit Test, Integration Test and E2E Test

Caso de uso real: Un carrito de compras: la prueba unitaria revisa que el total sea 200; la E2E simula a un cliente que entra, agrega un producto y paga.

Pregunta para pensar: ¿Qué es una regresión y cómo te protegen de ella las pruebas automáticas?

5.2 · Jest: describe, test, expect y matchers

Idea clave: Una prueba es una especificación que se ejecuta: «dado este escenario, espero este comportamiento».

Tipo	Video (clic para abrir)
Principal	Jest JS tutorial en español para principiantes – Curso de Testing
Refuerzo	Introducción al Testing desde cero con Jest
Buenas prácticas	¿Cómo se escribe una buena prueba unitaria en JavaScript? 10 buenas prácticas

Caso de uso real: Proteger carrito.js con cinco casos: carrito vacío, agregar, eliminar un producto inexistente, calcular el total y rechazar cantidades negativas (toThrow).

Pregunta para pensar: Si una prueba falla, ¿cambias la prueba o el código? ¿Cómo decides?

5.3 · Depuración profesional con Chrome DevTools

Idea clave: Más allá de console.log: detén el programa y observa Scope, Call Stack y Network con evidencia, no con suposiciones.

Tipo	Video (clic para abrir)
Principal	Basta de console.log(): domina el debugger del navegador paso a paso
Refuerzo	Curso de JavaScript #10: Debug y DevTools
Refuerzo	Depurar JavaScript con el navegador y con Visual Studio Code

Caso de uso real: Una lista que no muestra datos de la API: Network dice si la petición salió y qué respondió; un breakpoint muestra que el valor se usó antes de que la promesa terminara.

Pregunta para pensar: ¿Qué diferencia hay entre el síntoma (dónde ves el error) y la causa (dónde se originó)?

5.4 · Patrones de diseño: Singleton y Módulo

Idea clave: Un patrón es una solución conocida a un problema concreto, no una receta obligatoria.

Tipo	Video (clic para abrir)
Singleton	Patrón Singleton – Patrones de diseño – Programación en español
Singleton	#1 Singleton – Patrones de diseño con JavaScript
Módulo	Patrón Module en JavaScript: ejemplo paso a paso

Caso de uso real: Un gestor de notificaciones único: sin importar desde qué parte de la app se notifique, todo queda en el mismo historial. El módulo oculta el estado y sólo expone `notificar()` y `obtenerHistorial()`.

Pregunta para pensar: ¿Necesitas realmente una instancia única o sólo compartir comportamiento?

Cierre del curso: ¿cómo se conecta todo?

Día 1 entender JavaScript → **Día 2** organizar el código → **Día 3** coordinar lo asíncrono → **Día 4** construir interfaces → **Día 5** probar, depurar y estructurar. Testing permite comprobar; debugging, comprender; la arquitectura, evolucionar.

Proyecto integrador – Task Manager: IDs únicos con closures (Tema 1); clases, campos privados y al menos 3 módulos ESM (Tema 2); API con `fetch`, `async/await` y `try/catch` (Tema 3); delegación de eventos y `localStorage` (Tema 4); al menos 4 pruebas con Jest y depuración con DevTools (Tema 5). Presentación de 5 minutos con defensa técnica.

Siguientes pasos: sube tu código a un repositorio público, explora proyectos Open Source y convierte tus ejercicios en portafolio.